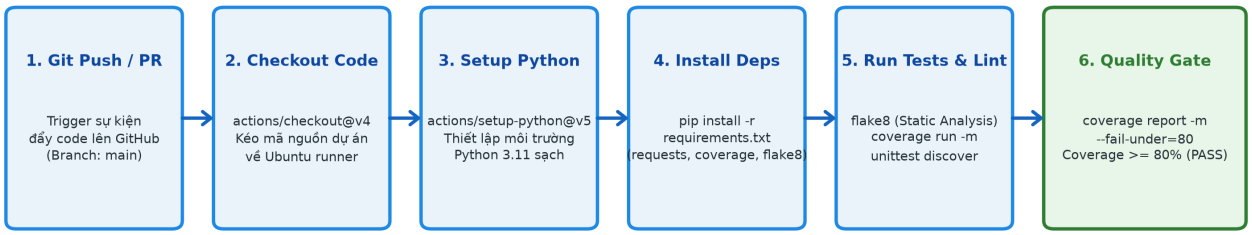
BÀI THỰC HÀNH 7 - MINH CHỨNG TỰ TAY THỰC HIỆN CỦA SINH VIÊN

Họ và tên: Nguyễn Hoài Nam | MSSV: 2312694 | Ngày thực hiện: 16/10/2026 | Chữ ký tươi: Nam (Nguyễn Hoài Nam)

MỤC: TRANG 1: SƠ ĐỒ LUỒNG KIẾN TRÚC CI PIPELINE (YÊU CẦU VIẾT TAY 1)

Mô tả bản vẽ luồng kiến trúc CI Pipeline với 6 khối tuần tự chuẩn:

SƠ ĐỒ KIẾN TRÚC ĐƯỜNG ỐNG CI/CD - YUMMYAPP MOBILE BACKEND



Phân tích chi tiết 6 khối kiến trúc trong luồng CI:

Khối 1: Git Push Event	Sự kiện kích hoạt (Trigger): Lập trình viên đẩy commit hoặc tạo PR vào nhánh main.
Khối 2: Checkout Code	Hành động actions/checkout@v4: Tải toàn bộ mã nguồn dự án về máy chủ ảo Ubuntu.
Khối 3: Setup Python 3.11	Hành động actions/setup-python@v5: Cấu hình môi trường Python 3.11 sạch và độc lập.
Khối 4: Install Dependencies	Lệnh shell: Cài đặt các thư viện đã ghim cứng trong requirements.txt.
Khối 5: Execute Unit Tests & Linter	Phân tích tĩnh cú pháp bằng flake8 và chạy toàn bộ test tự động bằng coverage run.
Khối 6: Quality Gate	Cổng chất lượng: Kiểm tra độ bao phủ ≥ 80% (--fail-under=80). Nếu đạt -> PASS (Xanh), nếu <80% -> Bị chặn Đỏ.

BÀI THỰC HÀNH 7 - MINH CHỨNG TỰ TAY THỰC HIỆN CỦA SINH VIÊN

Họ và tên: Nguyễn Hoài Nam | MSSV: 2312694 | Ngày thực hiện: 16/10/2026 | Chữ ký tươi: Nam (Nguyễn Hoài Nam)

MỤC: TRANG 2: BẢNG MA TRẬN THIẾT KẾ KỊCH BẢN TEST CASE MOCKING (YÊU CẦU VIẾT TAY 2)

Bảng kê ma trận thiết kế 7 kịch bản kiểm thử Cấp độ 2 (4 cột bắt buộc):

Tên Test Case	Hàm được gọi	Đối tượng & Giá trị Mocking	Kết quả kỳ vọng (Assert)
1. Online hợp lệ (test_apply_promo_online_valid)	apply_promo_code(total, 'GIAM10')	network_status = 'online' os_version = 15	Giảm 10% đơn hàng: total = 100k -> 90k
2. Offline ném lỗi (test_apply_promo_offline_raises_connection_error)	apply_promo_code(total, 'GIAM10')	network_status = 'offline'	Ném ConnectionError: 'Khong co ket noi mang de xac thuc ma.'
3. Mã đặc biệt trên OS 15 (test_apply_promo_special_code_valid_os15)	apply_promo_code(total, 'NEW_OS_ONLY')	network_status = 'online' os_version = 15	Giảm 20% đơn hàng: total = 200k -> 160k
4. Mã đặc biệt trên OS 14 ném lỗi (test_apply_promo_special_code_lower_os14_raises_value_error)	apply_promo_code(total, 'NEW_OS_ONLY')	network_status = 'online' os_version = 14	Ném ValueError: 'Ma nay yeu cau HDH phien ban 15 tro len.'
5. Mock GPS trong vùng (test_get_nearby_restaurants_in_range)	get_nearby_restaurants()	@patch('yummy_app.gps_service.get_current_location') return_value = (10.0, 105.0)	Trả về: ['Nha hang Pho Ly Quoc Su', 'Bun cha Huong Lien']
6. Mock GPS ngoài vùng (test_get_nearby_restaurants_out_of_range)	get_nearby_restaurants()	@patch('yummy_app.gps_service.get_current_location') return_value = (20.0, 110.0)	Trả về danh sách rỗng: []
7. Mock GPS mất tín hiệu (test_get_nearby_restaurants_gps_signal_lost)	get_nearby_restaurants()	@patch('yummy_app.gps_service.get_current_location') side_effect = GPSSignalLostError	Bắt ngoại lệ, trả về: 'Khong the xac dinh vi tri.'

BÀI THỰC HÀNH 7 - MINH CHỨNG TỰ TAY THỰC HIỆN CỦA SINH VIÊN

Họ và tên: Nguyễn Hoài Nam | MSSV: 2312694 | Ngày thực hiện: 16/10/2026 | Chữ ký tươi: Nam (Nguyễn Hoài Nam)

MỤC: TRANG 3: LỜI GIẢI VIẾT TAY CÂU HỎI PHÂN TÍCH CẤP ĐỘ 2 (YÊU CẦU VIẾT TAY 3)

Câu 2.1: Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

Lời giải viết tay chi tiết:

1. Nguyên lý Thất bại sớm (Fail-Fast): Linter thực hiện phân tích mã tĩnh (Static Code Analysis) để quét các lỗi cú pháp (SyntaxError), biến chưa khai báo, lỗi import. Quá trình này diễn ra cực kỳ nhanh (1-2 giây) và tốn rất ít tài nguyên máy chủ.
2. Tiết kiệm chi phí và tài nguyên CI: Bộ kiểm thử Unit Test đòi hỏi nạp môi trường, fixture, mock dịch vụ và tính toán coverage tốn nhiều thời gian. Nếu mã nguồn đã có lỗi cú pháp thì chắc chắn test suite sẽ gãy (crash). Đặt Linter chạy trước giúp hủy tiến trình ngay từ đầu, giải phóng runner của GitHub Actions.
3. Chuẩn hóa chất lượng mã nguồn: Đảm bảo toàn bộ mã đáp ứng đúng chuẩn hình thức và không có lỗi tĩnh trước khi đo lường độ bao phủ.

Câu 2.2: Tham số --fail-under=80 của lệnh coverage report hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?

Lời giải viết tay chi tiết:

1. Nguyên lý Mã thoát (Exit Code / Return Code): Trong môi trường Linux/POSIX, mọi tiến trình khi kết thúc đều trả về tiến trình cha một mã số nguyên (Exit Code) từ 0 đến 255. Quy ước chuẩn: Exit Code = 0 là thành công; Exit Code != 0 (1, 2,...) là có lỗi.
2. Cơ chế của coverage report --fail-under=80: Lệnh đo đạc tỷ lệ bao phủ mã nguồn. Nếu độ bao phủ thực tế < 80%, chương trình chủ động gọi hàm thoát trả về Exit Code khác 0 (thường là 2).
3. Tín hiệu ngắt tiến trình trên GitHub Actions: Runner thực thi shell với tùy chọn bash -e (errexit). Khi một câu lệnh trả về Exit Code khác 0, runner lập tức dừng toàn bộ job, đánh dấu bước đó màu ĐỎ (FAILED) và khóa đường ống không cho merge mã nguồn.

Câu 2.3: Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử khi tối ưu độ bao phủ mã nguồn?

Lời giải viết tay chi tiết:

1. Thông tin hiển thị: Tham số -m (tương đương --show-missing) thêm cột Missing vào bảng báo cáo, hiển thị cụ thể số dòng hoặc khoảng dòng mã lệnh chưa hề được thực thi (uncovered lines) trong quá trình chạy test.
2. Lợi ích đối với kỹ sư kiểm thử:

• Định vị chính xác điểm mù: Giúp kỹ sư biết ngay dòng code nào trong nhánh rẽ if/else hoặc khối except chưa được test.

• Tối ưu hóa thiết kế test case: Viết đúng test case có dữ liệu đầu vào hoặc mock trạng thái thích hợp để kích hoạt dòng mã thiếu, nhanh chóng đưa độ bao phủ vượt ngưỡng Quality Gate ≥ 80% mà không lãng phí công sức.

BÀI THỰC HÀNH 7 - MINH CHỨNG TỰ TAY THỰC HIỆN CỦA SINH VIÊN

Họ và tên: Nguyễn Hoài Nam | MSSV: 2312694 | Ngày thực hiện: 16/10/2026 | Chữ ký tươi: Nam (Nguyễn Hoài Nam)

MỤC: TRANG 4: LỜI GIẢI VIẾT TAY CÂU HỎI PHÂN TÍCH CẤP ĐỘ 3 (YÊU CẦU VIẾT TAY 4)

Câu 3.1: Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.

Lời giải viết tay chi tiết:

1. Flaky Test (Kiểm thử chập chờn / bất định):

• Bản chất: Test case chạy trên cùng một phiên bản mã và cùng môi trường cấu hình nhưng lúc Pass lúc Fail một cách ngẫu nhiên.

• Nguyên nhân: Do xung đột bất đồng bộ (Race condition), độ trễ mạng Internet không ổn định, gọi trực tiếp API bên thứ ba bị timeout, hoặc thiết bị phần cứng thật như GPS bị mất sóng tín hiệu ngẫu nhiên do vật cản thời tiết.

2. Lỗi do sai lệch môi trường phụ thuộc (Environment/Dependency Drift):

• Bản chất: Test chạy Pass trên máy lập trình viên (Local machine) nhưng lại thất bại nhất quán 100% trên máy chủ CI/CD.

• Nguyên nhân: Khác biệt phiên bản Python, khác biệt hệ điều hành (Windows vs Ubuntu), hoặc không ghim phiên bản dẫn đến việc CI tự động tải phiên bản thư viện mới chứa thay đổi không tương thích (Breaking Changes).

3. So sánh tóm tắt: Flaky test mang tính ngẫu nhiên (non-deterministic); sai lệch môi trường mang tính nhất quán (deterministic) trên môi trường bị lệch.

Câu 3.2: Tại sao trên máy chủ CI bắt buộc phải tuân thủ nguyên tắc ghim phiên bản (Version Pinning) trong requirements.txt?

Lời giải viết tay chi tiết:

1. Tính tái lập (Reproducibility) và Tính xác định (Determinism): Đảm bảo mọi lần chạy CI đều cài đúng phiên bản nhị phân đã được kiểm thử, giúp kết quả build phản ánh trung thực chất lượng mã nguồn chứ không bị tác động bởi các bản vá ngẫu nhiên bên ngoài.

2. Ngăn ngừa phá vỡ mã do cập nhật ngầm (Breaking Changes): Nếu không ghim phiên bản, pip sẽ tải bản mới nhất (latest). Khi thư viện bên ngoài đổi tên hàm hoặc thay đổi tham số, pipeline CI sẽ lập tức bị gãy (Break the build) dù lập trình viên không sửa code.

3. Bảo mật chuỗi cung ứng phần mềm: Kiểm soát chặt chẽ các gói thư viện, ngăn chặn nguy cơ CI tự động tải phiên bản mới bị chèn mã độc.

Câu 3.3: Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.

Lời giải viết tay chi tiết:

• Bước 1 - Tạo nhánh phụ (Feature/Chore Branch): Tạo nhánh riêng từ main (ví dụ: chore/upgrade-requests). Tuyệt đối không sửa trực tiếp trên main.

• Bước 2 - Đọc kỹ Release Notes & Changelog: Nghiên cứu tài liệu của thư viện để nắm các Breaking Changes, các hàm bị deprecate.

• Bước 3 - Cập nhật và kiểm thử cục bộ: Sửa số phiên bản trong requirements.txt, sửa mã nguồn nếu API thay đổi, chạy test và linter trên máy local.

• Bước 4 - Tạo Pull Request (PR) & Kích hoạt CI: Đẩy commit lên GitHub và tạo PR. GitHub Actions tự động kiểm định: chạy linter, unit test, quality gate.

• Bước 5 - Đánh giá mã nguồn (Code Review): Thành viên nhóm review diff, xác nhận CI tích xanh 100% không có lỗi hồi quy.

• Bước 6 - Hợp nhất an toàn (Merge PR): Merge PR vào main và theo dõi pipeline trên nhánh main hoàn thành mỹ mãn.